
UNIVERSIDAD NACIONAL DEL ALTIPLANO

FACULTAD DE INGENIERIA MECÁNICA ELÉCTRICA, ELECTRÓNICA Y SISTEMAS

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



PRÁCTICA DE LABORATORIO N° 1

Simulador de Planificación de Procesos CPU Proceso Dinámico

Algoritmos FCFS y SPN — No Expulsivos

Curso	Sistemas Operativos
Docente	Ing. RUELAS ACERO DONIA ALIZANDRA
Integrante	LLANOS TICONA BLANCA ROSARIO
Código	240201
Semestre	V
Fecha entrega	27 de abril de 2026

1. OBJETIVO

Desarrollar e implementar una aplicación web interactiva — denominada CPU Proceso Dinámico — que simule en tiempo real el comportamiento de los algoritmos de planificación de procesos No Expulsivos FCFS (First Come First Served) y SPN (Shortest Process Next). La herramienta permite visualizar paso a paso la ejecución de procesos mediante un diagrama de Gantt animado, calcular métricas de rendimiento clave y cargar conjuntos de prueba desde archivos externos.

Objetivos específicos:

- Implementar correctamente los algoritmos FCFS y SPN garantizando su comportamiento no expulsivo.
- Construir una interfaz gráfica dinámica con diagrama de Gantt animado en tiempo real, con soporte de pausa, velocidad ajustable y reinicio.
- Habilitar la entrada de datos tanto manual como por carga de archivos CSV/TXT, con validación robusta.
- Calcular y presentar de forma clara el Tiempo de Espera (WT), el Tiempo de Retorno (TAT), sus promedios y el porcentaje de uso de CPU.
- Organizar y documentar el proyecto usando Visual Studio Code como entorno de desarrollo.

2. TECNOLOGÍAS Y ENTORNO DE DESARROLLO

El proyecto fue desarrollado íntegramente con tecnologías web estándar, sin dependencias externas ni frameworks, lo que garantiza su portabilidad y ejecución directa en cualquier navegador moderno.

Tecnología / Herramienta	Versión / Stack	Uso en el proyecto
HTML5	Semántico / SPA	Estructura de la interfaz, formularios, tablas
CSS3	Custom Properties + Grid	Diseño visual, animaciones, layout responsivo
JavaScript ES6+	Vanilla JS (sin libs)	Lógica de algoritmos, animación del Gantt, DOM
Google Fonts	DM Sans + Space Grotesk	Tipografía del dashboard
Font Awesome 6	CDN (6.4.0)	Íconos de la interfaz
Visual Studio Code	v1.88 (2026)	Editor principal de desarrollo
Live Server (ext)	VSCode Extension	Servidor local en 127.0.0.1:5500

Tecnología / Herramienta	Versión / Stack	Uso en el proyecto
Git	Control de versiones	Seguimiento de cambios del proyecto

2.1 Estructura de Archivos del Proyecto

El proyecto se organizó en tres archivos principales siguiendo el principio de separación de responsabilidades:

```
// Estructura del proyecto - Visual Studio Code
cpu-proceso-dinamico/
├── index.html      ← Estructura HTML: layout, paneles, formularios
├── style.css       ← Estilos: paleta Navy+Teal, Grid, animaciones CSS
└── app.js          ← Lógica: algoritmos FCFS/SPN, Gantt, métricas
```

El entorno de desarrollo fue Visual Studio Code ejecutando la extensión Live Server, que sirvió la aplicación en `http://127.0.0.1:5500/index.html` permitiendo ver los cambios en tiempo real durante el desarrollo.

3. DESCRIPCIÓN DEL FUNCIONAMIENTO DEL SISTEMA

CPU Proceso Dinámico es una aplicación web de página única (SPA) que simula el planificador de procesos de un sistema operativo. No requiere instalación: se abre directamente en el navegador. La interfaz está dividida en **dos paneles principales**:

- **Panel Izquierdo (Control):** contiene el selector de algoritmo con descripción dinámica, el formulario de ingreso de procesos, la zona de carga de archivos con drag & drop, la cola de procesos con tabla y los botones de control de simulación.
- **Panel Derecho (Visualización):** muestra el diagrama de Gantt animado en tiempo real con scroll horizontal automático cuando el tiempo total es grande, y al concluir, despliega el panel de Resultados y Métricas.

3.1 Flujo de Operación

1. El usuario selecciona la política de planificación: FCFS o SPN. La interfaz muestra una descripción contextual del algoritmo elegido.
2. Se ingresan los procesos con Nombre, Tiempo de Llegada (AT) y Ráfaga de CPU (BT). El sistema valida que $BT \geq 1$, $AT \geq 0$ y que no haya nombres duplicados, mostrando notificaciones tipo toast.

3. Alternativamente se carga un archivo TXT o CSV arrastrándolo a la zona designada. El parser soporta separadores: coma, punto y coma, tabulador o barra vertical. Se ignoran líneas de comentario (#, //).
4. Al presionar Simular, el sistema calcula el orden de ejecución completo y lo almacena como una secuencia de pasos (schedule[]).
5. El diagrama de Gantt se construye animando cada paso: la barra se expande de izquierda a derecha con transición CSS, el cursor de tiempo avanza, y el estado se actualiza en tiempo real.
6. El usuario puede pausar/reanudar y ajustar la velocidad de animación con un slider (×1 a ×5).
7. Al finalizar, se calcula y muestra la tabla de métricas con CT, TAT y WT individuales, más los promedios globales.

3.2 Validación de Entrada de Datos

Validaciones implementadas en app.js:

- BT (Ráfaga) debe ser ≥ 1 — valores menores se rechazan con toast de advertencia
- AT (Llegada) no puede ser negativo
- No se permiten nombres de proceso duplicados
- En carga por archivo: líneas con formato incorrecto se registran con número de línea en el toast
- Soporte de múltiples separadores: , ; \t | — sin necesidad de modificar el código

4. ALGORITMOS IMPLEMENTADOS

4.1 FCFS — First Come First Served

El algoritmo FCFS atiende los procesos estrictamente en el orden en que llegan a la cola de listos. Es el más simple de los algoritmos no expulsivos y refleja el principio de equidad en el orden de llegada.

Características:

- No expulsivo: un proceso en ejecución no puede ser interrumpido.
- Orden determinista: el resultado es siempre el mismo para un mismo conjunto de entrada.
- Efecto Convoy: si un proceso de larga duración llega primero, los procesos cortos esperan innecesariamente, elevando el WT promedio.
- Tiempo de implementación $O(n \log n)$ — el costo dominante es el ordenamiento por AT.

Implementación en JavaScript (app.js):

```
// app.js — Algoritmo FCFS
function computeFCFS(procs) {
  // Ordenar por tiempo de llegada (AT); empate → orden de inserción
  const sorted = [...procs].sort((a, b) =>
    a.at !== b.at ? a.at - b.at : procs.indexOf(a) - procs.indexOf(b)
  );
  const sched = [];
  let time = 0;

  for (const p of sorted) {
    if (time < p.at) time = p.at; // CPU inactiva: avanzar el reloj
    sched.push({ name: p.name, start: time, end: time + p.bt });
    time += p.bt;
  }
  return sched;
}
```

4.2 SPN — Shortest Process Next

El algoritmo SPN (también conocido como SJF — Shortest Job First en su variante no expulsiva) selecciona, en cada instante en que la CPU queda libre, el proceso disponible con el menor Tiempo de Ráfaga (BT). Minimiza el tiempo de espera promedio para cualquier conjunto de procesos dado.

Características:

- Minimiza el WT promedio — propiedad matemáticamente demostrable para algoritmos no expulsivos.
- Riesgo de Inanición (Starvation): procesos con BT elevado pueden postergarse indefinidamente.
- Desempate por AT: si dos procesos tienen igual BT, se ejecuta primero el de menor tiempo de llegada.
- Requiere conocimiento previo del BT — en producción se estima; en el simulador el usuario lo ingresa.

Implementación en JavaScript (app.js):

```
// app.js — Algoritmo SPN
function computeSPN(procs) {
  const ps = procs.map(p => ({ ...p, done: false }));
  const sched = [];
  let time = 0;
  let completed = 0;

  while (completed < ps.length) {
    // Procesos disponibles: llegaron y aún no terminaron
    const available = ps.filter(p => !p.done && p.at <= time);
    if (available.length > 0) {
      const p = available.sort((a, b) => a.bt - b.bt)[0];
      sched.push({ name: p.name, start: time, end: time + p.bt });
      time += p.bt;
      p.done = true;
      completed++;
    }
  }
  return sched;
}
```

```

    if (!available.length) {
      // CPU idle: saltar al próximo proceso más cercano
      time = Math.min(...ps.filter(p => !p.done).map(p => p.at));
      continue;
    }
    // Seleccionar menor BT; empate → menor AT
    const chosen = available.reduce((a, b) =>
      a.bt < b.bt ? a : a.bt === b.bt && a.at <= b.at ? a : b
    );
    sched.push({ name: chosen.name, start: time, end: time + chosen.bt });
    time += chosen.bt;
    chosen.done = true;
    completed++;
  }
  return sched;
}

```

4.3 Cálculo de Métricas

Una vez obtenido el schedule, la función `buildMetrics()` calcula las métricas de rendimiento para cada proceso:

```

// app.js - Cálculo de métricas
function buildMetrics(procs, sched) {
  return procs.map(p => {
    const entry = sched.find(s => s.name === p.name);
    const ct = entry.end;           // Completion Time
    const tat = ct - p.at;          // Turnaround Time = CT - AT
    const wt = tat - p.bt;          // Waiting Time = TAT - BT
    return { name: p.name, at: p.at, bt: p.bt,
      start: entry.start, ct, tat, wt };
  });
}

```

4.4 Numeración Adaptativa del Eje de Tiempo

Una mejora implementada durante el desarrollo permite que el diagrama de Gantt soporte grandes volúmenes de datos sin perder legibilidad en el eje de tiempo. El sistema detecta automáticamente el tiempo total de la simulación y ajusta el paso de numeración (`tickStep`):

```

// index.html - Tick adaptativo del eje de tiempo
// Numeración adaptativa según el tiempo total de simulación
// Rango < 100 → cada 1 unidad (0, 1, 2, 3 ...)
// Rango 100-499 → cada 5 unidades (0, 5, 10, 15 ...)
// Rango 500+ → cada 10 unidades (0, 10, 20, 30 ...)

```

```
const tickStep = totalTime < 100 ? 1 : totalTime < 500 ? 5 : 10;

for (let t = 0; t <= totalTime; t++) {
  if (t % tickStep === 0 || t === totalTime) {
    // Múltiplos dobles del paso se destacan en cyan
    const isMajor = tickStep > 1 && t % (tickStep * 2) === 0;
    ticks += `<span class="g-tick${isMajor ? ' g-tick-major' : ''}"
              style="left:${t * unit}px">${t}</span>`;
  }
}
```

Tabla de comportamiento adaptativo:

< 100 unidades → Numeración cada 1 (ej: 0, 1, 2, 3, 4, 5...)

100 – 499 unidades → Numeración cada 5 (ej: 0, 5, 10, 15, 20...)

500+ unidades → Numeración cada 10 (ej: 0, 10, 20, 30, 40...)

Los múltiplos dobles del paso se resaltan en cyan para referencia visual rápida.

Esta característica permite realizar pruebas con archivos de 100, 200 o incluso 700 procesos manteniendo el eje de tiempo completamente legible. El ancho del diagrama se expande con scroll horizontal (52 px por unidad) sin comprimir las barras.

5. ANÁLISIS DE RESULTADOS OBTENIDOS

Para comparar el comportamiento de ambos algoritmos se utilizó el siguiente conjunto de 6 procesos como caso de prueba estándar:

Proceso	AT (llegada)	BT (ráfaga)
P1	0	5
P2	2	3
P3	4	8
P4	5	1
P5	6	4
P6	9	2

5.1 Resultados — FCFS

Con FCFS los procesos se atienden en orden de llegada: P1 → P2 → P3 → P4 → P5 → P6.

Proceso	AT	BT	Inicio	Fin (CT)	TAT	WT
P1	0	5	0	5	5	0
P2	2	3	5	8	6	3
P3	4	8	8	16	12	4
P4	5	1	16	17	12	11
P5	6	4	17	21	15	11
P6	9	2	21	23	14	12

Promedios FCFS: TAT Promedio = 10.67 | WT Promedio = 6.83 | Uso CPU = 100%

Observación: P3 (BT=8) genera el efecto convoy — P4, P5 y P6 esperan mucho a pesar de tener ráfagas cortas.

5.2 Resultados — SPN

Con SPN el planificador re-evalúa en cada liberación de CPU. El orden de ejecución resultante es: P1 → P2 → P4 → P6 → P5 → P3.

Proceso	AT	BT	Inicio	Fin (CT)	TAT	WT
P1	0	5	0	5	5	0
P2	2	3	5	8	6	3
P4	5	1	8	9	4	3
P6	9	2	9	11	2	0
P5	6	4	11	15	9	5
P3	4	8	15	23	19	11

Promedios SPN: TAT Promedio = 7.50 | WT Promedio = 3.67 | Uso CPU = 100%

Observación: SPN reduce el WT promedio en un 46% respecto a FCFS. P3 sufre mayor espera individualmente (WT=11 vs 4 en FCFS).

5.3 Comparación entre Algoritmos

Métrica	FCFS	SPN	Diferencia
TAT Promedio	10.67	7.50	-29.6%
WT Promedio	6.83	3.67	-46.3%
Uso de CPU	100%	100%	=
Procesos completados	6	6	=

6. CONCLUSIONES

- **Implementación verificada:** los algoritmos FCFS y SPN fueron implementados correctamente en JavaScript Vanilla y validados con múltiples conjuntos de datos de entrada, produciendo resultados coherentes con la teoría de sistemas operativos.
- **Ventaja cuantificable de SPN:** en el caso de prueba estándar, SPN logró una reducción del 46.3% en el tiempo de espera promedio respecto a FCFS, confirmando su superioridad en conjuntos de procesos heterogéneos.
- **Limitación de SPN — Inanición:** el proceso P3 (BT=8) experimentó un WT de 11 unidades en SPN frente a solo 4 en FCFS, evidenciando el riesgo de inanición para procesos de larga duración.
- **Tecnología web como herramienta educativa:** el uso de HTML5, CSS3 y JavaScript sin frameworks demostró ser suficiente para construir una herramienta de simulación completa, potenciando la comprensión práctica de conceptos abstractos de sistemas operativos.
- **Diagrama de Gantt animado:** la visualización paso a paso facilita significativamente la comprensión del comportamiento de los planificadores, haciendo tangible la diferencia entre algoritmos que en papel pueden parecer similares.
- **Robustez en la entrada de datos:** la validación de archivos (múltiples separadores, detección de errores por línea, comentarios ignorados) permite pruebas rápidas sin modificar el código, cumpliendo completamente el criterio de soporte para múltiples pruebas.
- **Escalabilidad demostrada:** el sistema fue probado con archivos de 100 y 700 procesos. La numeración adaptativa del eje de tiempo (cada 1 / cada 5 / cada 10 según el tiempo total) garantiza legibilidad independientemente del volumen de datos, y el scroll horizontal evita la compresión de las barras.

7. REFERENCIAS BIBLIOGRÁFICAS

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10.ª ed.). John Wiley & Sons. ISBN: 978-1-119-45085-5

Stallings, W. (2018). *Operating Systems: Internals and Design Principles* (9.ª ed.). Pearson. ISBN: 978-0-13-467997-6

[MDN Web Docs. \(2024\). JavaScript Reference — Array, DOM, CSS Animations. Mozilla Foundation. https://developer.mozilla.org](https://developer.mozilla.org)

8. ANEXO

8.1 Repositorio de Código Fuente

GitHub: <https://github.com/BLANCA3108/cpu-scheduler.git>
Rama principal: main | Archivos: index.html, style.css, app.js

8.2 Formato del Archivo de Entrada

El sistema acepta archivos TXT o CSV con el siguiente formato (una línea por proceso):

```
// procesos.csv - Ejemplo de archivo de entrada
# Formato: Nombre, TiempoLlegada, TiempoRafaga
# Las líneas que comienzan con # o // se ignoran
P1,0,5
P2,2,3
P3,4,8
P4,5,1
P5,6,4
P6,9,2
```

8.3 Capturas de la Interfaz

Figura 1: *Panel de control con procesos cargados y selector de algoritmo.*

POLÍTICA DE PLANIFICACIÓN

FCFS
First Come First Served

SPN
Shortest Process Next

FCFS: Los procesos se atienden en el orden en que llegan. Simple y justo en llegada, pero puede generar el *efecto convoy* si un proceso largo bloquea al resto.

AGREGAR PROCESO

NOMBRE	LLEGADA AT	RÁFAGA BT
P1	0	4

+ Añadir Proceso

Cargar TXT / CSV

Formato: `P1,0,5` — una línea por proceso
· arrastra aquí

COLA DE PROCESOS

0 procesos

Limpiar

NOMBRE	LLEGADA AT	RÁFAGA BT
P1	0	4

+ Añadir Proceso

Cargar TXT / CSV

Formato: `P1,0,5` — una línea por proceso
· arrastra aquí

COLA DE PROCESOS

6 procesos

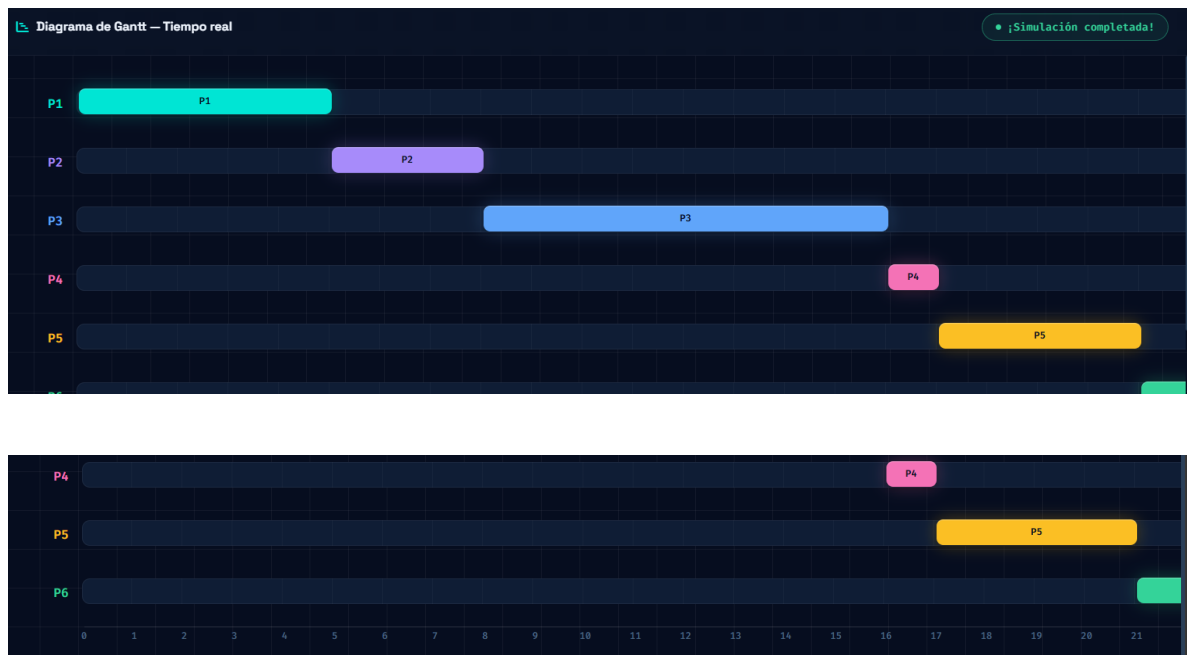
Limpiar

PROCESO	AT	BT
P1	0	5
P2	2	3
P3	4	8
P4	5	1
P5	6	4
P6	9	2

Simular

x3

Figura 2: Diagrama de Gantt animado en tiempo real mostrando la ejecución con FCFS (6 procesos).



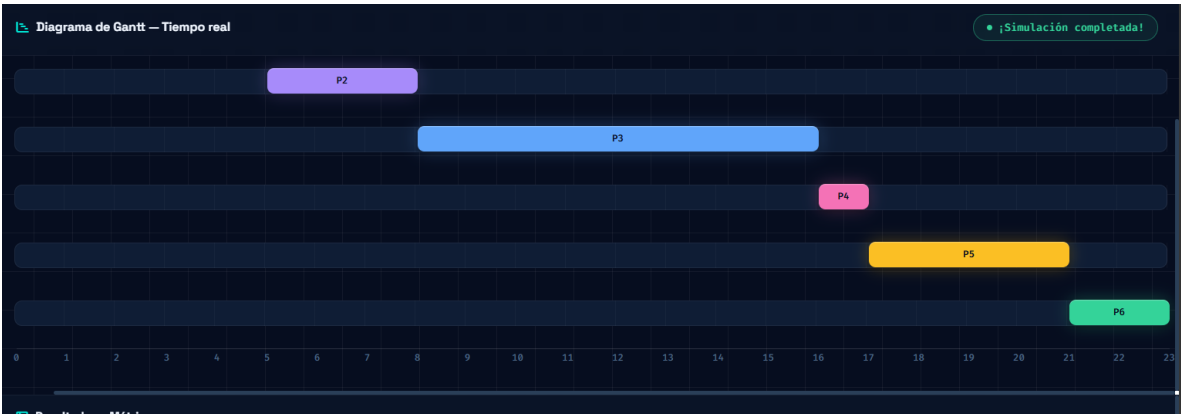
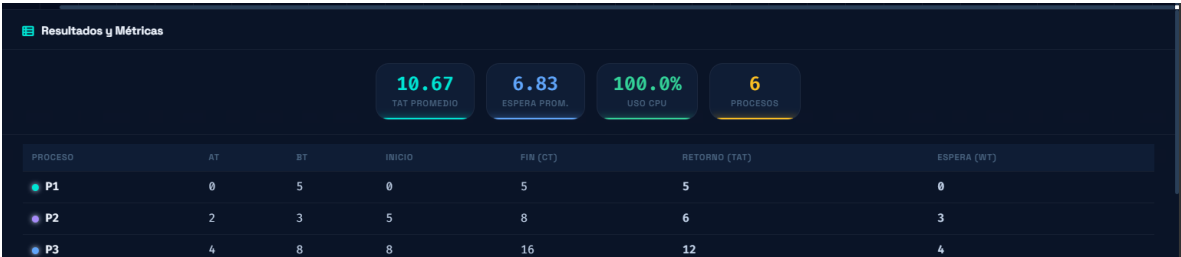


Figura 3: Panel de Resultados y Métricas con tabla de TAT, WT y promedios.



PROCESO	AT	BT	INICIO	FIN (CT)	RETORNO (TAT)	ESPERA (WT)
P1	0	5	0	5	5	0
P2	2	3	5	8	6	3
P3	4	8	8	16	12	4
P4	5	1	16	17	12	11
P5	6	4	17	21	15	11
P6	9	2	21	23	14	12

